

Data Structures: Assignment-8

Name: Suraj Shah

Admission Number: I24AI012

Question 1: Implement a priority queue using a linked list. The queue has the following basic operations:

- enqueue() – Insert an element in the queue.
- dequeue() – Remove the element from the queue.
- peek() – Return the element at the front node of the queue
- isFull() – Validates if the queue is full or not.
- isEmpty() – Checks if the queue is empty.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    int pri;
    struct Node *next;
}Node;

void enqueue(Node **head, int pri, int data) {
    Node *newNode=(Node*)malloc(sizeof(Node));
    newNode->pri=pri;
    newNode->data=data;
    newNode->next=NULL;
    if (*head==NULL) {
        *head=newNode;
    }

    else if((*head)->pri<pri) {
        newNode->next=*head;
        *head=newNode;
        return;
    }

    else {
        Node *temp=*head;

        while (temp->next!=NULL && temp->next->pri>pri) {
```

```

        temp=temp->next;
    }

    newNode->next=temp->next;
    temp->next=newNode;
    return;
}

int isEmpty(Node *head) {
    if (head==NULL) {
        return 1;
    }
    return 0;
}

int dequeue(Node **head) {
    if (isEmpty(*head)) {
        printf("Queue is Empty\n");
        return -1;
    }
    Node *temp=*head;
    printf("%d deleted with priority %d", temp->data, temp->pri);
    *head=(*head)->next;
    free(temp);
}

int peek(Node **head) {
    if (isEmpty(*head)) {
        printf("Queue is Empty\n");
        return -1;
    }
    printf("The front element of the queue is %d", (*head)->data);
}

int main() {
    Node *head=NULL;
    int choice, data, pri;
    while (1) {
        printf("\nPriority Queue Operations:\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Peek\n");
        printf("0. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:

```

```

        printf("Enter data: ");
        scanf("%d", &data);
        printf("Enter priority: ");
        scanf("%d", &pri);
        enqueue(&head, pri, data);
        break;
    case 2:
        dequeue(&head);
        break;
    case 3:
        peek(&head);
        break;
    case 0:
        return 0;

    default:
        printf("Invalid choice!\n");
    }
}
}

```

Output:

```

Priority Queue Operations:
1. Enqueue
2. Dequeue
3. Peek
0. Exit
Enter your choice: 1
Enter data: 10
Enter priority: 1

```

```

Priority Queue Operations:
1. Enqueue
2. Dequeue
3. Peek
0. Exit
Enter your choice: 1
Enter data: 20
Enter priority: 2

```

```

Priority Queue Operations:
1. Enqueue
2. Dequeue
3. Peek
0. Exit
Enter your choice: 1
Enter data: 30
Enter priority: 3

```

```

Priority Queue Operations:
1. Enqueue
2. Dequeue
3. Peek
0. Exit
Enter your choice: 2
30 deleted with priority 3
Priority Queue Operations:
1. Enqueue
2. Dequeue
3. Peek
0. Exit
Enter your choice: 3
The front element of the queue is 20

```

Question 2: Implement a Circular queue using a linked list. The queue has the following basic operations:

- enqueue() – Insert an element in the queue.
- dequeue() – Remove the element from the queue.
- peek() – Return the element at the front node of the queue
- isFull() – Validates if the queue is full or not.
- isEmpty() – Checks if the queue is empty.

Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
}Node;

void enqueue(Node **f, Node **r, int data) {
    Node *newNode=(Node*)malloc(sizeof(Node));
    newNode->data=data;
    newNode->next=NULL;
    if (*r==NULL) {
        *r=newNode;
        *f=newNode;
    }
    else {
        (*r)->next=newNode;
        *r=newNode;
        newNode->next=*f;
    }
}

int isEmpty(Node *f) {
    if (f==NULL) {
        return 1;
    }
    return 0;
}

int dequeue(Node **f, Node **r) {
    Node *temp=*f;
    if (isEmpty(*f)) {
```

```

        printf("Queue is Empty\n");
        return -1;
    }

    else {
        (*r)->next=(*f)->next;
        *f=(*f)->next;
        printf("Element deleted: %d", temp->data);
        free(temp);
    }
}

int peek(Node **f, Node **r) {
    if (isEmpty(*f)) {
        printf("Queue is Empty\n");
        return -1;
    }
    print("The front element of the queue is %d", (*f)->data);
}

int main() {
    Node *front=NULL;
    Node *rear=NULL;
    int choice, data;
    while (1) {
        printf("\nQueue Operations:\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Peek\n");
        printf("0. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter data: ");
                scanf("%d", &data);
                enqueue(&front, &rear, data);
                break;
            case 2:
                dequeue(&front, &rear);
                break;
            case 3:
                peek(&front, &rear);
                break;
            case 0:
                return 0;

            default:

```

```
        printf("Invalid choice!\n");
    }
}
```

Output:

Queue Operations:

1. Enqueue
2. Dequeue
3. Peek
0. Exit

Enter your choice: 1

Enter data: 10

Queue Operations:

1. Enqueue
2. Dequeue
3. Peek
0. Exit

Enter your choice: 1

Enter data: 20

Queue Operations:

1. Enqueue
2. Dequeue
3. Peek
0. Exit

Enter your choice: 1

Enter data: 30

Queue Operations:

1. Enqueue
2. Dequeue
3. Peek
0. Exit

Enter your choice: 2

Element deleted: 10

Queue Operations:

1. Enqueue
2. Dequeue
3. Peek
0. Exit

Enter your choice: 3

The front element of the queue is 20